

Piloto de una Red de Monitoreo y Alerta Temprana de Huaicos basada en Nodos IoT con Conectividad Híbrida (Terrestre-Satelital) en la Cuenca del Río Rímac.

"Año de la Esperanza y el Fortalecimiento de la Democracia"



UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ingeniería Electrónica y Eléctrica

AVANCE DEL PROYECTO

Curso:

Redes de Computadoras I

Docente:

VILLAFUERTE BARRETO, HERNAN OSWALDO

Sección

G1

Integrantes:

Shiguay Vargas Illary Ariadna

Arteaga Ayca Adrian

2026

1. Introducción

El presente entregable desarrolla el marco teórico y los requerimientos técnicos del proyecto **“Piloto de una red de monitoreo y alerta temprana de huaicos basada en nodos IoT con conectividad híbrida terrestre-satelital en la cuenca del río Rímac”**. A partir del problema planteado en el Entregable 1, se profundizan los conceptos de redes de computadoras que sustentan la propuesta, tales como el modelo OSI, TCP/IP, VLAN, STP, direccionamiento IP, subneteo, enrutamiento, métricas de red y análisis de protocolos.

El proyecto propone una red simulada en GNS3 donde un nodo sensor IoT transmite telemetría hacia un broker MQTT ubicado en una red central. Para ello, se considera una arquitectura compuesta por switches, routers, VLANs, enlaces redundantes y nodos Alpine Linux. Esta estructura permite aplicar conceptos de Capa 2, Capa 3 y Capa de Aplicación dentro de un escenario relacionado con monitoreo ambiental y alerta temprana.

Asimismo, se definen los requerimientos funcionales y no funcionales del prototipo, junto con los valores objetivo que servirán como referencia para los entregables posteriores. Estos valores permitirán evaluar la red en términos de latencia, throughput, jitter, pérdida de paquetes, convergencia STP y disponibilidad.

2. Marco teórico aplicado al escenario

2.1 Modelo OSI y TCP/IP

El modelo OSI permite analizar una red en siete capas: física, enlace de datos, red, transporte, sesión, presentación y aplicación. En el proyecto, las capas más relevantes son la capa de enlace, la capa de red, la capa de transporte y la capa de aplicación.

En la **capa de enlace de datos**, se ubican tecnologías como Ethernet, VLAN y STP. Esta capa permite la transmisión de tramas dentro de una red local, la identificación de equipos mediante direcciones MAC y el control de bucles cuando existen enlaces redundantes entre switches.

En la **capa de red**, se emplea el protocolo IP para direccionar los dispositivos y permitir la comunicación entre redes diferentes. En el prototipo, esta capa se representa mediante subredes IPv4, routers R1 y R2, y comunicación entre la red del sensor IoT y la red central donde se ubica el broker MQTT. IPv4 fue definido originalmente como un protocolo para transportar datagramas entre hosts identificados por direcciones lógicas en redes interconectadas.

En la **capa de transporte**, se utiliza TCP para permitir la comunicación confiable entre el sensor y el broker MQTT. TCP es un protocolo de transporte importante dentro de la pila de Internet y proporciona comunicación orientada a conexión.

Finalmente, en la **capa de aplicación**, se ubica MQTT, protocolo utilizado para enviar telemetría desde el sensor IoT hacia el broker Mosquitto. MQTT es un protocolo ligero de publicación/suscripción usado en escenarios IoT y M2M.

En relación con TCP/IP, el proyecto puede resumirse así:

Capa TCP/IP	Aplicación en el proyecto
Aplicación	MQTT, Mosquitto, telemetría IoT
Transporte	TCP, puerto 1883
Internet	IPv4, direccionamiento, enrutamiento
Acceso a red	Ethernet, VLAN, STP, ARP

2.2 Capa de enlace: VLAN, STP/RSTP y agregación de enlaces

2.2.1 VLAN IEEE 802.1Q

Una VLAN permite dividir una red física en varias redes lógicas independientes. Esto significa que varios dispositivos pueden estar conectados a la misma infraestructura de switches, pero pertenecer a dominios de difusión diferentes. En el proyecto se propone el uso de tres VLANs:

VLAN	Nombre	Función
VLAN 10	Sensores_IoT	Transporte de telemetría IoT
VLAN 20	Camaras_Video	Tráfico de video IP
VLAN 30	Gestion_Red	Administración de equipos

El estándar IEEE 802.1Q especifica el funcionamiento de puentes MAC con VLAN, incluyendo protocolos, administración y algoritmos asociados. En el proyecto, esto permite separar el tráfico crítico de sensores IoT del tráfico de video y gestión. Esta segmentación reduce la propagación de broadcast y mejora la organización lógica de la red.

2.2.2 Spanning Tree Protocol

STP es un protocolo de Capa 2 utilizado para evitar bucles en redes con enlaces redundantes. Cuando dos switches están conectados por más de un camino, existe el riesgo de que las tramas Ethernet circulen indefinidamente, generando tormentas de broadcast y degradación de la red.

En el prototipo, STP se aplica entre ESW1 y ESW2. La idea es mantener un enlace principal en estado **Forwarding** y otro enlace redundante en estado **Blocking**. Si el enlace principal falla, STP permite que el enlace alternativo se active para mantener la conectividad.

En el proyecto se considera STP como mecanismo de redundancia básica ante fallas de enlace. No se plantea como reemplazo de soluciones avanzadas de alta disponibilidad, sino como una forma de demostrar el control de bucles y la recuperación ante caída de un enlace en la capa de acceso.

2.2.3 RSTP

RSTP es una evolución de STP que reduce el tiempo de convergencia en redes con enlaces redundantes. Sin embargo, en la implementación del proyecto se trabajó principalmente con STP tradicional en los switches Cisco EtherSwitch de GNS3. Por ello, RSTP se considera como referencia teórica y posible mejora futura, pero no como el protocolo principal de la simulación.

2.2.4 Agregación de enlaces

La agregación de enlaces consiste en combinar varios enlaces físicos para que funcionen como un enlace lógico de mayor capacidad o disponibilidad. Esta técnica puede emplearse para mejorar el ancho de banda y ofrecer redundancia entre switches o routers.

En este proyecto, la agregación de enlaces se considera como una mejora futura. La simulación se enfoca principalmente en STP para demostrar redundancia ante falla de enlace, pero en una implementación física se podría utilizar agregación de enlaces para mejorar el rendimiento entre switches de acceso y distribución.

2.3 Capa de red: IPv4, IPv6, subneteo y enrutamiento

2.3.1 Direccionamiento IPv4

IPv4 permite identificar equipos dentro de una red mediante direcciones de 32 bits. En el proyecto, IPv4 se utiliza para asignar direcciones a los nodos principales: sensor IoT, broker MQTT, gateways y enlace entre routers.

El esquema propuesto es el siguiente:

Red		Subred	Uso
Red	de	192.168.10.0/24	Sensor IoT

sensores		
Red de video	192.168.20.0/24	Cámaras o tráfico multimedia
Red de gestión	192.168.30.0/24	Administración
Red central	192.168.40.0/24	Broker MQTT
Enlace R1-R2	10.0.0.0/30	Interconexión entre routers

El sensor IoT se ubica en la red 192.168.10.0/24, mientras que el broker MQTT se ubica en la red 192.168.40.0/24. La comunicación entre ambas redes requiere encaminamiento mediante routers.

2.3.2 IPv6

IPv6 es la versión más reciente del Protocolo de Internet y fue estandarizada en RFC 8200. Su principal ventaja frente a IPv4 es el mayor espacio de direcciones, además de mejoras en estructura de encabezado y soporte para redes modernas.

En el proyecto, IPv6 se considera de forma teórica porque la simulación principal se desarrolla con IPv4. Sin embargo, en una implementación futura de mayor escala, IPv6 podría facilitar la identificación directa de múltiples sensores IoT distribuidos en la cuenca del río Rímac.

2.3.3 Subneteo CIDR/VLSM

CIDR permite representar redes mediante una notación de prefijo, como /24 o /30. En el proyecto, las redes de usuarios se plantean con prefijo /24, lo que permite hasta 254 direcciones útiles por red. Esto es suficiente para una simulación académica y deja espacio para agregar más sensores, cámaras o equipos de gestión.

Para el enlace entre routers se propone una subred /30, ya que un enlace punto a punto solo requiere dos direcciones útiles. Esto evita desperdiciar direcciones IPv4.

Tabla de justificación:

Segmento	Subred	Máscara	Justificación
Sensores IoT	192.168.10.0/24	255.255.255.0	Permite crecimiento de sensores
Video IP	192.168.20.0/24	255.255.255.0	Segmenta tráfico multimedia

Gestión	192.168.30.0/24	255.255.255.0	Aísla administración
Broker MQTT	192.168.40.0/24	255.255.255.0	Red central de monitoreo
R1-R2	10.0.0.0/30	255.255.255.252	Enlace punto a punto

2.3.4 Enrutamiento estático y dinámico

El enrutamiento permite que paquetes de distintas redes lleguen a su destino. En el proyecto, la comunicación entre el sensor y el broker requiere que R1 y R2 conozcan las rutas hacia las redes 192.168.10.0/24 y 192.168.40.0/24.

Para una simulación inicial, el enrutamiento estático es suficiente porque la topología es pequeña y controlada. Sin embargo, en una red real con más sedes, sensores o enlaces alternativos, sería recomendable evaluar protocolos dinámicos como OSPF. En este entregable, el enfoque se mantiene en la definición teórica y técnica de la comunicación entre redes.

2.4 Métricas de red

Las métricas permiten evaluar cuantitativamente el desempeño del prototipo. En este proyecto se consideran latencia, throughput, jitter, pérdida de paquetes y disponibilidad.

2.4.1 Latencia

La latencia es el tiempo que tarda un paquete en viajar desde un origen hasta un destino. En pruebas con ping, normalmente se mide el RTT o tiempo de ida y vuelta.

Fórmula general:

Latencia promedio = suma de RTT medidos / número de paquetes recibidos

En el proyecto, se evaluará la latencia entre el sensor IoT y el broker MQTT.

2.4.2 Throughput

El throughput es la tasa efectiva de datos transmitidos exitosamente por unidad de tiempo. Se mide en bps, kbps o Mbps.

Fórmula:

Throughput = bits transmitidos / tiempo de transferencia

En el prototipo, esta métrica permitirá estimar la capacidad de la red para transportar tráfico desde el sensor hacia el broker.

2.4.3 Jitter

El jitter es la variación del retardo entre paquetes consecutivos. Es importante para analizar estabilidad en comunicaciones en tiempo real o tráfico sensible al retardo.

Fórmula aproximada:

Jitter promedio = suma de $|\text{RTT actual} - \text{RTT anterior}|$ / número de diferencias

En el proyecto, se utilizará para evaluar la variación de latencia durante pruebas de ping extendido.

2.4.4 Pérdida de paquetes

La pérdida de paquetes indica el porcentaje de paquetes enviados que no llegaron al destino.

Fórmula:

Pérdida (%) = $((\text{paquetes enviados} - \text{paquetes recibidos}) / \text{paquetes enviados}) \times 100$

Esta métrica es importante porque un sistema de alerta temprana debe minimizar la pérdida de mensajes críticos.

2.4.5 Disponibilidad

La disponibilidad mide el porcentaje de tiempo en que el sistema se mantiene operativo.

Fórmula:

Disponibilidad (%) = $(\text{tiempo operativo} / \text{tiempo total}) \times 100$

En el proyecto, la disponibilidad se relaciona con la capacidad de mantener comunicación ante fallas de enlace usando STP.

2.5 Análisis de protocolos

2.5.1 ARP

ARP permite asociar una dirección IPv4 con una dirección MAC dentro de una red local. RFC 826 describe ARP como un método para convertir direcciones de protocolo, como IP, en direcciones locales, como direcciones Ethernet.

En el proyecto, ARP es necesario para que el sensor IoT pueda comunicarse con su gateway dentro de la VLAN 10.

2.5.2 ICMP

ICMP se utiliza para enviar mensajes de control y diagnóstico dentro de redes IP. RFC 792 indica que ICMP se emplea para reportar errores en el procesamiento de datagramas.

En el prototipo, ICMP será utilizado mediante pruebas de **ping** para validar conectividad entre el sensor y el broker MQTT.

2.5.3 TCP

TCP es un protocolo orientado a conexión que proporciona comunicación confiable entre aplicaciones. En este proyecto es relevante porque MQTT funciona sobre TCP, normalmente en el puerto 1883. TCP fue especificado como protocolo de transporte importante dentro de la pila de Internet.

2.5.4 UDP

UDP es un protocolo de transporte no orientado a conexión. RFC 768 define UDP como un protocolo que permite enviar datagramas en redes interconectadas usando IP como protocolo subyacente.

En el proyecto, UDP no es el transporte principal de MQTT, pero puede aparecer en servicios auxiliares como DNS o DHCP.

2.5.5 DHCP

DHCP permite asignar direcciones IP automáticamente a los equipos de una red. En el prototipo final se plantea principalmente direccionamiento estático para mantener control sobre las IPs de sensores, routers y broker. Sin embargo, DHCP podría utilizarse en una ampliación futura para asignar direcciones a dispositivos no críticos.

2.5.6 DNS

DNS permite resolver nombres de dominio en direcciones IP. En el proyecto, la comunicación principal se realiza mediante direcciones IP directas, como **192.168.40.10**. Sin embargo, en una red real se podría usar DNS para identificar el broker mediante un nombre como **broker.coen.local**.

3. Requerimientos funcionales

Los requerimientos funcionales describen las funciones que debe cumplir el sistema de red.

Código Requerimiento funcional

- RF-01 El sistema debe permitir comunicación entre el sensor IoT y el broker MQTT.
- RF-02 El sistema debe segmentar el tráfico mediante al menos dos VLANs.
- RF-03 La red debe separar el tráfico de sensores IoT, video IP y gestión.
- RF-04 El sensor IoT debe enviar telemetría hacia el broker MQTT usando el protocolo MQTT.
- RF-05 El broker MQTT debe recibir mensajes publicados en tópicos definidos.
- RF-06 La red debe permitir pruebas ICMP entre nodos principales.
- RF-07 Los switches deben ejecutar STP para evitar bucles de Capa 2.
- RF-08 La red debe mantener conectividad ante la falla de un enlace redundante.
- RF-09 El tráfico debe poder ser capturado y analizado mediante Wireshark.
- RF-10 El direccionamiento IPv4 debe permitir comunicación entre la red de sensores y la red central.

4. Requerimientos no funcionales

Los requerimientos no funcionales definen condiciones de calidad, rendimiento y seguridad básica del sistema.

Código Requerimiento no funcional

- RNF-01 La red debe presentar una latencia inter-red menor a 100 ms en condiciones normales.
- RNF-02 La pérdida de paquetes debe ser menor al 5% en condiciones normales.
- RNF-03 El jitter máximo esperado debe ser menor a 20 ms.
- RNF-04 La red debe contar con segmentación lógica para reducir tráfico innecesario.
- RNF-05 El diseño debe ser escalable para agregar más sensores o cámaras.

RNF-0 El sistema debe permitir análisis de protocolos con Wireshark.
6

RNF-0 La red debe usar direccionamiento ordenado y documentado.
7

RNF-0 La disponibilidad objetivo del prototipo debe ser de 99%.
8

RNF-0 El tráfico crítico de sensores debe estar separado del tráfico de video y gestión.
9

RNF-1 La implementación debe poder reproducirse en un entorno de simulación GNS3.
0

5. Tecnología emergente: IoT y MQTT

La tecnología emergente seleccionada para el proyecto es **Internet de las Cosas (IoT)**, utilizando **MQTT** como protocolo de comunicación.

IoT permite conectar sensores y dispositivos físicos para recopilar información del entorno y transmitirla hacia sistemas de monitoreo. En el proyecto, el sensor IoT representa un nodo ubicado en la cuenca del río Rímac, encargado de generar datos de telemetría relacionados con el nivel del río o el estado de alerta.

MQTT funciona bajo el modelo publicación/suscripción. En este modelo, el sensor actúa como **publicador**, enviando datos hacia un tópico específico. El broker Mosquitto recibe el mensaje y lo entrega a los clientes suscritos. OASIS define MQTT como un protocolo ligero, abierto y adecuado para escenarios restringidos, lo que lo hace apropiado para aplicaciones IoT.

En el prototipo, el flujo teórico es:

Sensor IoT → VLAN 10 → R1 → R2 → Broker MQTT

El broker se ubica en la red central con la dirección:

192.168.40.10

El tópico propuesto para las pruebas es:

rimac/vlan10/sensor01/nivel

El mensaje puede tener un formato similar a:

```
{"sensor":"S01","nivel_cm":85,"estado":"normal"}
```

Este primer acercamiento teórico permite justificar la integración de MQTT en los siguientes entregables, donde se validará con capturas de Wireshark y pruebas de comunicación.

6. Métricas objetivo del prototipo

En este entregable se definen los valores objetivo que servirán como referencia para evaluar la red en las siguientes etapas.

Métrica	Valor objetivo	Justificación
Tiempo de convergencia STP	< 3 s	Recuperación rápida ante falla de enlace
Latencia intra-VLAN	< 20 ms	Comunicación local entre sensor y gateway
Latencia inter-VLAN/inter-red	< 100 ms	Comunicación sensor-broker aceptable
Throughput mínimo	0.3 Mbps	Suficiente para mensajes MQTT pequeños
Pérdida de paquetes máxima	< 5%	Mantener confiabilidad de telemetría
Jitter máximo	< 20 ms	Evitar variación excesiva en retardo
Disponibilidad objetivo	99%	Mantener operación estable del sistema

Estos valores son estimaciones iniciales. En los entregables posteriores serán comparados con mediciones reales obtenidas mediante **ping**, tráfico TCP, fallas de enlace, congestión y capturas en Wireshark.

7. Conclusión parcial

El marco teórico desarrollado permite sustentar técnicamente el diseño del prototipo de red propuesto. La segmentación mediante VLAN permite organizar el tráfico de sensores, video y gestión; STP permite controlar enlaces redundantes y evitar bucles; IPv4 permite direccionar los nodos principales; y los protocolos ARP, ICMP, TCP, UDP, DHCP y DNS permiten comprender el funcionamiento de la comunicación en diferentes capas.

Asimismo, la tecnología emergente IoT/MQTT se adapta al escenario de monitoreo ambiental, ya que permite transmitir telemetría ligera desde un sensor hacia un broker central. Los requerimientos funcionales, no funcionales y métricas objetivo definidos en este entregable servirán como base para el diseño detallado, la simulación y la experimentación cuantitativa de los siguientes entregables.

8. Referencias preliminares

- [1] J. Postel, “Internet Protocol,” RFC 791, IETF, 1981.
- [2] S. Deering y R. Hinden, “Internet Protocol, Version 6 (IPv6) Specification,” RFC 8200, IETF, 2017.
- [3] IEEE Standards Association, “IEEE 802.1Q-2018: Bridges and Bridged Networks,” 2018.
- [4] D. Plummer, “An Ethernet Address Resolution Protocol,” RFC 826, IETF, 1982.
- [5] J. Postel, “Internet Control Message Protocol,” RFC 792, IETF, 1981.
- [6] W. Eddy, “Transmission Control Protocol,” RFC 9293, IETF, 2022.
- [7] J. Postel, “User Datagram Protocol,” RFC 768, IETF, 1980.
- [8] OASIS Open, “MQTT Version 5.0,” OASIS Standard, 2019.